Line-by-Line Explanation of the Maple Code for the Key Vanishing Lemma (Logarithmic Case)

May 9, 2026

1 Introduction

This Maple code proves a special case of the Key Vanishing Lemma for the logarithmic setting, which asserts that certain negatively twisted invariant 2-jet differentials on the complement of a generic plane curve vanish identically. The present computation treats a curve $\mathcal{C} \subset \mathbb{P}^2$ of degree 13, with weighted order $m = 8$ and twist $t = 7$ (i.e. $\mathcal{O}_{\mathbb{P}^2}(-7)$). The algorithm forces all coefficients of a candidate jet differential to be zero by systematically imposing holomorphicity conditions, first on the affine charts and then across the line at infinity.

Flexibility of parameters. The three essential numbers are set at the very beginning:

`m` (weighted order), `twist` (the negative twist), `d` (curve degree minus one).

For a curve of arbitrary degree D one takes `d` $= D - 1$, because the defining equation is written as $y^D + \cdots$. The user can freely adjust `m, twist, d` to cover all cases required by Lemma 1.2 for curves of degree 12 and 13. The present run with $m = 8$, `twist` $= -7$, $d = 12$ serves as a concrete example.

Why degree 11 cannot be treated by this code. The method relies on the fact that the curve is a perturbation of the Fermat curve, so that R_y is simply a power of y. For a curve of degree 11 (i.e. $d = 10$) with a similar equation $y^{11} = P(x)$, a non-zero invariant logarithmic 2-jet differential exists for the parameters $(m, t) = (3, 1)$. This non-vanishing is detected by the same algorithm; therefore the linear systems constructed here would admit non-trivial solutions for $d = 10$, and the code would *not* produce a vanishing statement.

We now proceed line by line through the Maple code.

2 Initialization

```
restart:
m := 8;
twist := -7;
d := 12;                    degre := d + 1:
```

- `restart`: clears all previous definitions.
- `m := 8`: the weighted order of the 2-jet differential. It corresponds to the integer m in the bundle $E_{2,m}T^*_{\mathbb{P}^2}(\log \mathcal{C})$.
- `twist := -7`: the negative twist; the bundle is tensored with $\mathcal{O}_{\mathbb{P}^2}(-7)$. The code later uses this value to determine admissible degrees of the coefficient polynomials.
- `d := 12`: the auxiliary integer; the true curve degree will be $d + 1 = 13$.
- `degre := d + 1`: stores the actual degree, although this variable is not used later.

3 The homogeneous equation and its affine form

```
HSS := T^13 + T^6*X^7 + X^13:
SS := subs({T=1,X=x}, HSS);
```

- `HSS` defines the homogeneous polynomial $\widehat{R}(T,X,Y) = T^{13} + T^6X^7 + X^{13} + Y^{13}$. Note that the term Y^{13} is added later in `HRR`. Here only the part involving T and X is given.
- `SS` substitutes $T = 1$, $X = x$ into `HSS`, giving the affine polynomial $1 + x^{13} + x^7$. This will be part of the equation $R(x, y) = 0$.

4 Template of the invariant 2-jet differential

```
Jet_xR := expand(
+ (1/Ry^(m-0*2)) * add(AA||j * x1^(m-0*3-j) * (R1/RR)^j * Delta_xR^0, j=0..m-0*3)
+ (1/Ry^(m-1*2)) * add(BB||j * x1^(m-1*3-j) * (R1/RR)^j * Delta_xR^1, j=0..m-1*3)
+ (1/Ry^(m-2*2)) * add(CC||j * x1^(m-2*3-j) * (R1/RR)^j * Delta_xR^2, j=0..m-2*3)
+ (1/Ry^(m-3*2)) * add(DD||j * x1^(m-3*3-j) * (R1/RR)^j * Delta_xR^3, j=0..m-3*3)
+ (1/Ry^(m-4*2)) * add(EE||j * x1^(m-4*3-j) * (R1/RR)^j * Delta_xR^4, j=0..m-4*3) ):
```

This constructs the general local form of an invariant logarithmic 2-jet differential on the chart $\{R_y \neq 0\}$ (Proposition 4.2 in the paper):

$$J_{xR} = \sum_{k=0}^{\lfloor m/3 \rfloor} \frac{1}{R_y^{m-2k}} \sum_{j=0}^{m-3k} F_{j,k}(x,y)\,(x')^{\,m-3k-j}\left(\tfrac{R'}{R}\right)^j (\Delta'''_{xR})^k.$$

- `x1` and `x2` denote x' and x''.
- `R1` and `R2` denote R' and R'' (derivatives of R along the curve).
- `RR` is the affine equation $R(x, y) = 0$ (to be defined later).
- `Delta_xR` is the logarithmic Wronskian.
- The coefficients `AA||j`, `BB||j`, `CC||j`, `DD||j`, `EE||j` are undetermined polynomials in x and y.

The sums run over j from 0 to $m-3k$; when $m-3k<0$ the `add` is empty. For $m = 8$ the non-empty blocks are $k = 0, \ldots, 2$ (since $8 - 3\cdot 2 = 2 \geq 0$, while $8 - 3\cdot 3 = -1$). Consequently $k = 0, 1, 2$ contribute, while $k = 3, 4$ are empty.

5 Degree bounds for the unknown polynomials

```
for l from 0 to m - 0*3 do dA[l] := (m-0*2)*d - m + l*1 + 0*1 + twist: od:
for l from 0 to m - 1*3 do dB[l] := (m-1*2)*d - m + l*1 + 1*1 + twist: od:
for l from 0 to m - 2*3 do dC[l] := (m-2*2)*d - m + l*1 + 2*1 + twist: od:
for l from 0 to m - 3*3 do dD[l] := (m-3*2)*d - m + l*1 + 3*1 + twist: od:
for l from 0 to m - 4*3 do dE[l] := (m-4*2)*d - m + l*1 + 4*1 + twist: od:
```

Each coefficient $F_{j,k}$ will be written as a polynomial in x and y. The total degree of the homogenized coefficient must equal

$$\deg \widehat{F}_{j,k} = (m-2k)(\deg R_y) - m + j + k + \texttt{twist},$$

with $\deg R_y = d$ (in our notation $R_y = (d+1)y^d$, so its homogeneous degree is d). These loops assign the maximal exponent of x or y to `dA[l]`, `dB[l]`, etc. They will be used later to restrict the monomial expansion of each $F_{j,k}$.

6 The Wronskian and the change of variables

```
Delta_xR := x1 * (R2/RR - R1*R1/RR^2) - x2 * (R1/RR):
```

This is the logarithmic Wronskian on the chart $\{R_y \neq 0\}$:

$$\Delta'''_{xR} = x' \left(\frac{R''}{R} - \frac{(R')^2}{R^2} \right) - x'' \frac{R'}{R}.$$

```
Solx1 := - y1*Ry/Rx + R1/Rx:
Solx2 := - Rxx*y1^2*Ry^2/Rx^3 + 2*Rxx*y1*Ry*R1/Rx^3 - Rxx*R1^2/Rx^3
         + 2*y1^2*Rxy*Ry/Rx^2 - 2*y1*Rxy*R1/Rx^2 - y1^2*Ryy/Rx - y2*Ry/Rx + R2/Rx:
```

Here `y1` and `y2` stand for y' and y''. The formulas express x' and x'' in terms of y', y'', R', R'' and the partial derivatives of R. They are obtained by solving the linear system

$$R' = x' R_x + y' R_y, \qquad R'' = x'' R_x + y'' R_y + (x')^2 R_{xx} + 2x'y' R_{xy} + (y')^2 R_{yy}.$$

7 Transition to the other affine chart

```
Jet_Ry := sort(mtaylor(
  expand(subs(R2=-(RR^2*Delta_Ry-RR*R1*y2-R1^2*y1)/(RR*y1),
  expand(subs({x1=Solx1,x2=Solx2}, Jet_xR)))),
  [R1,y1,Delta_Ry], 100), [R1,y1,Delta_Ry]):
```

- `subs({x1=Solx1,x2=Solx2}, Jet_xR)` replaces x', x'' by their expressions in terms of y', y'', R', R''.
- Then `R2` is replaced using the relation

  $$R'' = -\frac{R^2 \Delta'''_{Ry} - R\,R'y'' - (R')^2 y'}{R\,y'},$$

  which comes from the definition of the Wronskian on the other chart. This substitution rewrites everything in terms of R', y', and Δ'''_{Ry} (the Wronskian on $\{R_x \neq 0\}$).
- The result is expanded and then developed as a multivariate Taylor series in $[R1, y1, Delta\_Ry]$ up to order 100 (more than sufficient) to obtain a polynomial expression that is then sorted.

Thus `Jet_Ry` is the expression of the same jet differential on the chart $\{R_x \neq 0\}$.

8 Extract the coefficients of the transformed expression

```
printlevel := 2:
for k from 0 to m/3 do
for j from 0 to m-3*k do
  EEq[m-j-3*k,j,k] := factor(
    Ry^(m-j-3*k)*RR^(m-j-3*k)*Rx^(m-2*k)
    * coeftayl(Jet_Ry, [R1,y1,Delta_Ry]=[0,0,0], [m-j-3*k,j,k]) ):
od: od:
```

For each triple (i, j, k) such that $i + j + 3k = m$ (here i is the exponent of R', j of y', k of Δ'''_{Ry}), `coeftayl` picks the coefficient of the monomial $(R')^i (y')^j (\Delta'''_{Ry})^k$ in `Jet_Ry`. The factor

$$R_y^i \, R^i \, R_x^{m-2k}$$

is multiplied to clear denominators that were allowed on the first chart but must disappear for the jet differential to be holomorphic on $\{R_x \neq 0\}$. The result `EEq[i,j,k]` is a polynomial in x and y.

9 Define the explicit curve and its partial derivatives

```
RR  := y^(d+1) + SS;
HRR := Y^(d+1) + HSS;
RY := diff(HRR, Y):
```

Now `RR` becomes the affine equation

$$R(x,y) = y^{13} + x^{13} + x^7 + 1.$$

`HRR` is the full homogeneous polynomial $\widehat{R}(T,X,Y) = T^{13} + T^6X^7 + X^{13} + Y^{13}$. `RY` is its partial derivative with respect to Y, i.e. $\widehat{R}_Y = 13\,Y^{12}$.

```
Rx := diff(RR,x):
Ry := diff(RR,y):
Rxx := diff(RR,x,x):
Rxy := diff(RR,x,y):
Ryy := diff(RR,y,y):
```

These commands compute all first and second order partial derivatives of the affine equation R.

10 Prepare the equations by truncating in y

```
printlevel := 2:
for k from 0 to m/3 do
for j from 0 to m-3*k do
  Eq[m-j-3*k,j,k] := mtaylor(EEq[m-j-3*k,j,k], y, (m-j-3*k)*d):
  Var[m-j-3*k,j,k] := indets(Eq[m-j-3*k,j,k]) minus {x,y}:
od: od:
```

Because $R_y = 13\,y^{12}$ and the curve is Fermat-like, the divisibility condition $R_y^i \mid$ something becomes $y^{12i} \mid$ something. Thus the polynomial `EEq` must be divisible by y^{id} (with $d = 12$, $i = m - j - 3k$). Equivalently, its Taylor expansion in y must have no terms of degree $< id$. `mtaylor(..., y, (m-j-3*k)*d)` truncates at that degree. The truncated polynomial is stored in `Eq[i,j,k]`, and the free variables in it are recorded.

11 Iterative solving of the divisibility conditions by powers of y

The core of the algorithm is an induction on $\ell = 1, 2, \ldots, 22$. At step ℓ, we require that all truncated equations `Eq`$[i,j,k]$ with $i = \ell$ be divisible by $y^{\ell d}$. Since the equations have already been truncated at order id, this means all their coefficients must vanish. We expand the unknown polynomials $F_{j,k}$ up to degree $\ell d - 1$ in y, collect the coefficients of $y^0, \ldots, y^{\ell d-1}$ from the relevant `Eq`'s, and solve the resulting linear system for the undetermined coefficients.

After each solving step, the solutions are globally assigned to the unknowns, thus reducing the number of free parameters. The process is repeated until the number of terms in each $F_{j,k}$ becomes 1 (meaning the polynomial is forced to be zero).

First iteration ($\ell = 1$): divisibility by y^{1d}

```
for l from 0 to m - 0*3 do
  AA||l := add(add(A||l[i,j]*x^i*y^j, i=0..dA[l]-j), j=0..min(dA[l],1*d-1)): od:
for l from 0 to m - 1*3 do
  BB||l := add(add(B||l[i,j]*x^i*y^j, i=0..dB[l]-j), j=0..min(dB[l],1*d-1)): od:
for l from 0 to m - 2*3 do
  CC||l := add(add(C||l[i,j]*x^i*y^j, i=0..dC[l]-j), j=0..min(dC[l],1*d-1)): od:
for l from 0 to m - 3*3 do
  DD||l := add(add(D||l[i,j]*x^i*y^j, i=0..dD[l]-j), j=0..min(dD[l],1*d-1)): od:
for l from 0 to m - 4*3 do
  EE||l := add(add(E||l[i,j]*x^i*y^j, i=0..dE[l]-j), j=0..min(dE[l],1*d-1)): od:
```

The coefficients $\texttt{AA}_j, \texttt{BB}_j, \ldots$ are expanded as sums of monomials $A_j[p,q]x^p y^q$ with $0 \le q \le \ell d - 1$ and $p+q$ bounded by the previously computed degrees. At this stage $\ell = 1$, so $q \le 11$.

```
Systemey1d := {}:
for k from 0 to (m-1)/3 do
  Coeffs_y1d[1,m-1-3*k,k] := {coeffs(expand(mtaylor(Eq[1,m-1-3*k,k], y, 1*d)), [x,y])}:
  Systemey1d := Systemey1d union Coeffs_y1d[1,m-1-3*k,k]:
od:
Systeme_y1d := Systemey1d:
```

For $i = 1$ (i.e. those $\texttt{Eq}[1,j,k]$ with $1 + j + 3k = m$), we recover all coefficients with respect to x and y after expanding again up to y^d. These coefficients must vanish. They are collected into the set `Systemey1d`.

```
Variables_y1d := indets(Systeme_y1d):
Solutions_y1d := solve(Systeme_y1d, Variables_y1d):
assign(Solutions_y1d):
```

The linear system is solved symbolically, and the solutions are assigned. Afterwards the number of terms in each coefficient polynomial is printed; they are still large (several thousands), showing that many parameters remain.

Iterations $\ell = 2, 3, \ldots, 22$

The same block of code is repeated for $\ell = 2, \ldots, 22$. In each iteration:

- The coefficients are re-expanded up to $y^{\ell d - 1}$.
- The equations for $i = \ell$ are truncated and their coefficients collected.
- The system is solved and the solutions are assigned.

The numbers of terms (`nops`) decrease step by step. At $\ell = 22$ all `nops` become 1, meaning every coefficient polynomial has collapsed to a single term, which, together with the previous equations, forces that term to be zero.

12 Checking holomorphicity at infinity

After all y-divisibility conditions are satisfied, the jet differential is holomorphic on the affine charts. It remains to ensure that it extends holomorphically across the line at infinity $\{T = 0\}$.

```
for l from 0 to m - 0*3 do
  HA||l := add(add(A||l[i,j]*X^i*Y^j*T^(dA[l]-i-j), i=0..dA[l]-j), j=0..dA[l]):
od:
...
for j from 0 to m - 0*3 do HK[j,0] := HA||j: od:
...
```

The coefficients are homogenized to obtain polynomials $\widehat{K}_{j,k}(T, X, Y)$ of the correct degree. They are stored in `HK[j,k]`.

```
printlevel := 2:
for q from 0 to m/3 do
for p from 0 to m-3*q do
  HEq[p,q] := expand(mtaylor(add(add(add(
    (-1)^k * 2^(p-r) * (d+1)^(j+k-p-q) * binomial(j,r) *
    (k!)/((p-r)!*q!*(k+r-p-q)!) *
    X^(m-q-j-2*k) * T^(k-q) * RY^(2*k-2*q) * HK[j,k],
    j=r..m-3*k),
    r=max(0,p+q-k)..min(p,m-3*k)),
    k=q..m/3), T, m-p-3*q)):
od: od:
```

This code implements the explicit transformation formula (see equation (64) in the paper) that describes the behaviour of the coefficients when passing from the chart $\{X \neq 0\}$ to $\{T \neq 0\}$. The resulting `HEq[p,q]` must be divisible by T^{m-3q-p}. The inner sums implement the combinatorial expansions, and `mtaylor(..., T, m-p-3q)` truncates the series in T at the required order, keeping only terms that must vanish.

```
HSysteme := {seq(seq(coeffs(HEq[p,q],[T,X,Y]),p=0..m-3*q),q=0..m/3)} minus {0}:
HSolutions := solve(HSysteme):
assign(HSolutions):
```

All coefficients of the truncated `HEq` are collected into a linear system, which is then solved. The solutions are assigned, removing any further degrees of freedom.

13 Final verification

```
Jet_xR := factor(
+ (1/Ry^(m-0*2)) * x^(-twist) * add(AA||j * x1^(m-0*3-j) * (R1/RR)^j * Delta_xR^0,
    j=0..m-0*3)
+ (1/Ry^(m-1*2)) * x^(-twist) * add(BB||j * x1^(m-1*3-j) * (R1/RR)^j * Delta_xR^1,
    j=0..m-1*3)
+ (1/Ry^(m-2*2)) * x^(-twist) * add(CC||j * x1^(m-2*3-j) * (R1/RR)^j * Delta_xR^2,
    j=0..m-2*3)
+ (1/Ry^(m-3*2)) * x^(-twist) * add(DD||j * x1^(m-3*3-j) * (R1/RR)^j * Delta_xR^3,
    j=0..m-3*3)
+ (1/Ry^(m-4*2)) * x^(-twist) * add(EE||j * x1^(m-4*3-j) * (R1/RR)^j * Delta_xR^4,
    j=0..m-4*3) ):
```

The jet differential is recomposed with the now fully determined coefficients. The extra factor $x^{-\texttt{twist}} = x^7$ accounts for the twist by $\mathcal{O}_{\mathbb{P}^2}(-7)$.

```
Vars := indets(Jet_xR) minus {x, x1, x2, y, y1, y2, R1, R2};
NbVars := nops(Vars);
```

`Vars` collects all remaining undetermined symbols (the A, b, C, D, E constants). The output shows `NbVars = 0`, proving that every coefficient has been forced to zero. Hence the only global section of the twisted invariant jet bundle is the trivial one.

14 Conclusion

The computation verifies

$$H^0\big(\mathbb{P}^2,\ E_{2,8}T^*_{\mathbb{P}^2}(\log \mathcal{C}) \otimes \mathcal{O}_{\mathbb{P}^2}(-7)\big) = 0$$

for the specific Fermat-type curve of degree 13. By semi-continuity, the same vanishing holds for a generic smooth curve of degree 13. Adjusting the initial parameters `m`, `twist` and `d` yields verification of all required cases for curves of degree 12 and 13 stated in Lemma 1.2. The method is not applicable to degree 11 because for $d = 11$ and $(m, t) = (3, 1)$ the algorithm would find a non-zero solution, reflecting the existence of a nontrivial negatively twisted jet differential for that degree.